

Code Smell Myths

Some code smells aren't really code smells at all. Programming is full of half-remembered bits of bad advice that are taken out of context or stick around long after they've outlived their usefulness. I blame tech book authors who try to pass off their subjective opinions as best practices.

You might have been told some of these practices are code smells, but they're mostly fine. I call them *code smell myths*: they're warnings that you can and should ignore. Let's look at a few of them.

Myth: Functions Should Have Only One return Statement at the End

The “one entry, one exit” idea comes from misinterpreted advice from the days of programming in assembly and FORTRAN languages. These languages allowed you to enter a subroutine (a structure similar to a function) at any point, including in the middle of it, making it hard to debug which parts of the subroutine had been executed. Functions don't have this problem (execution always begins at the start of the function). But the advice lingered, becoming “functions and methods should only have one `return` statement, which should be at the end of the function or method.”

Trying to achieve a single `return` statement per function or method often requires a convoluted series of `if-else` statements that's far more confusing than having multiple `return` statements. Having more than one `return` statement in a function or method is fine.

Myth: Functions Should Have at Most One try Statement

“Functions and methods should do one thing” is good advice in general. But taking this to mean that exception handling should occur in a separate function goes too far. For example, let's look at a function that indicates whether a file we want to delete is already nonexistent:

```
>>> import os
>>> def deleteWithConfirmation(filename):
...     try:
...         if (input('Delete ' + filename + ', are you sure? Y/N') == 'Y'):
```